# Robótica con Arduino y micro:bit

Nombre: _____ Fecha: _____

## Qué vas a aprender

- Conocer las placas programables Arduino y micro:bit y sus partes
- Montar circuitos sencillos con LED, pulsador y resistencia en una placa de prototipos
- Programar las funciones básicas setup y loop y la lectura y escritura de pines
- Programar la micro:bit con MakeCode usando sus sensores integrados

## Para pensar

**¿Cómo puede una placa del tamaño de una tarjeta de crédito controlar un robot, una estación meteorológica o una instalación artística?**

### Explicación

## Placas programables

Un **microcontrolador** es un chip que contiene procesador, memoria y entradas y salidas. A diferencia de un ordenador, ejecuta un único programa una y otra vez, desde que se enciende hasta que se apaga.

- **Arduino** nació en Italia en 2005 como proyecto de **hardware libre**: sus diseños son públicos y cualquiera puede fabricar placas compatibles. Se programa en un lenguaje basado en C++ o con bloques.
- **micro:bit** fue creada por la BBC en 2015 para que los estudiantes aprendieran a programar. Lleva sensores y luces ya integrados y se programa con bloques, JavaScript o Python.

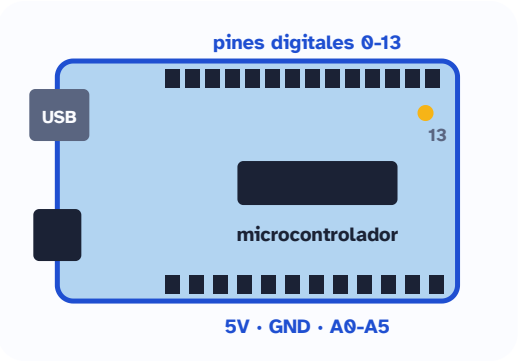
undefined

---

## La placa Arduino UNO

Es el modelo más usado en las aulas. Sus partes principales:

| Parte | Función |
|---|---|
| **Microcontrolador** (ATmega328P) | el «cerebro»; guarda y ejecuta el programa |
| **Puerto USB** | conecta al ordenador para cargar el programa y alimentar la placa |
| **Conector de alimentación** | pila o fuente de 7-12 V para funcionar sin ordenador |
| **Pines digitales 0-13** | entradas o salidas de 0 o 5 V; algunos (~) imitan valores intermedios (PWM) |
| **Pines analógicos A0-A5** | leen tensiones de 0 a 5 V como números de 0 a 1023 |
| **Pines 5V, 3.3V y GND** | alimentación para sensores; GND es el negativo (tierra) |
| **LED integrado (pin 13)** | permite probar sin montar nada |



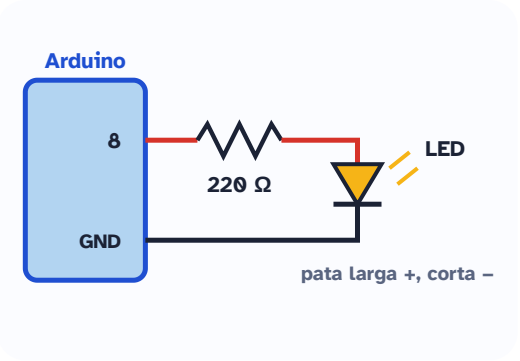

## La protoboard y el primer circuito

La **placa de prototipos** (*protoboard*) permite montar circuitos sin soldar. Sus agujeros están conectados por dentro: en las **columnas centrales**, de cinco en cinco; en los **carriles laterales**, a lo largo (para + y –).

Para encender un LED desde Arduino:

1. Cable del **pin 8** a una **resistencia de 220 Ω**.
2. La resistencia, a la **pata larga** del LED (ánodo, +).
3. La **pata corta** (cátodo, –), a **GND**.

La resistencia es imprescindible: limita la corriente para que el LED no se queme. Monta siempre con la placa **desconectada** y revisa antes de enchufar.




## Programar Arduino: setup y loop

Un programa de Arduino (llamado **sketch**) tiene siempre dos partes:

- `setup()`: se ejecuta una sola vez al encender. Se configuran los pines.
- `loop()`: se repite sin fin. Aquí va lo que la placa hace continuamente.

```
void setup() {
  pinMode(8, OUTPUT);     // el pin 8 será una salida
```

```
}
void loop() {
  digitalWrite(8, HIGH);  // enciende el LED (5 V)
  delay(500);             // espera 500 ms
  digitalWrite(8, LOW);   // apaga el LED (0 V)
  delay(500);
}
```

Este es el famoso «Blink»: el LED parpadea una vez por segundo. Se escribe en el **Arduino IDE** y se carga con el botón «Subir». También hay entornos de bloques, como **Arduinoblocks** o mBlock.

undefined

---

### Leer entradas: pulsadores y sensores

Para leer un **pulsador**, el pin se configura como entrada y se usa `digitalRead`, que devuelve `HIGH` (pulsado) o `LOW`:

```
void setup() {
  pinMode(2, INPUT_PULLUP);   // pulsador en el pin 2
  pinMode(8, OUTPUT);
}
void loop() {
  if (digitalRead(2) == LOW) digitalWrite(8, HIGH);
  else digitalWrite(8, LOW);
}
```

Con `INPUT_PULLUP`, el pin está a 5 V y baja a 0 V al pulsar, por eso pulsado es `LOW`. Para sensores analógicos se usa `analogRead(A0)`, que da un número de 0 a 1023. Con el **monitor serie** (`Serial.println`) se ven los valores en el ordenador: es la mejor herramienta para calibrar y depurar.

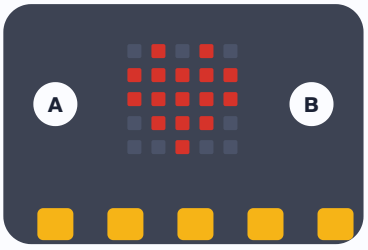
undefined

---

### La placa micro:bit

La **micro:bit** trae casi todo integrado, así que se puede programar sin montar nada:

- Una **matriz de 25 LED** (5 × 5) que muestra iconos, texto y números, y que también mide la luz.
- **Dos botones** (A y B) y un logotipo táctil.
- **Acelerómetro** (detecta agitar e inclinar), **brújula** y sensor de **temperatura**.
- **Radio** y Bluetooth para comunicar varias placas.
- Altavoz y micrófono (en la versión 2).
- Un **conector de borde** con pines para añadir motores y sensores externos.



25 LED · 2 botones · sensores integrados

## Programar la micro:bit con MakeCode

**MakeCode** es el editor web de Microsoft para la micro:bit. Funciona con bloques (como Scratch) y permite ver el mismo programa en JavaScript o Python. Incluye un **simulador** para probar sin la placa.

- **al iniciar**: equivale al `setup()` de Arduino.
- **para siempre**: equivale al `loop()`.
- Eventos: **al presionar el botón A**, **al agitar**, **al recibir por radio**…

Ejemplo, un dado electrónico: «al agitar → mostrar número (número al azar entre 1 y 6)». Para pasar el programa, se descarga un archivo `.hex` y se arrastra a la micro:bit, que el ordenador ve como una memoria USB.

undefined

## Del circuito al robot

Con una placa, dos **motores** con ruedas, un **controlador de motores**, una batería y uno o dos sensores ya tenemos un robot móvil:

- **Robot que esquiva**: ultrasonidos delante; si hay un obstáculo cerca, gira.
- **Robot siguelíneas**: dos sensores infrarrojos mirando al suelo; si uno ve negro, corrige hacia ese lado.
- **Robot teledirigido**: una micro:bit en la mano envía por radio la inclinación y otra en el robot mueve los motores.

Recomendaciones de seguridad: alimentar los motores aparte, no cortocircuitar la batería y desconectarla al modificar el montaje.

undefined

## Arduino o micro:bit: ¿cuál elegir?

| | **Arduino UNO** | **micro:bit** |
|---|---|---|
| Programación | C++ (IDE) o bloques | bloques, JavaScript, Python |
| Sensores integrados | ninguno (solo un LED) | luz, temperatura, acelerómetro, brújula, botones |
| Montaje | protoboard y componentes | casi nada; pinzas de cocodrilo |
| Ventaja | muy versátil, enorme comunidad, hardware libre | empezar en minutos, ideal para aprender |

Las dos comparten la misma lógica: leer entradas, decidir con un programa y controlar salidas. Hay además placas como **ESP32** (con wifi) o **Raspberry Pi**, que es ya un ordenador completo.

undefined

## Robótica con placas programables

| | |
|---|---|
| **Microcontrolador** | chip que ejecuta un programa sin parar |
| **Arduino** | pines 0-13 y A0-A5 · setup() y loop() |
| **micro:bit** | 25 LED · botones · acelerómetro · MakeCode |
| **Circuito LED** | pin → 220 Ω → LED (pata larga) → GND |
| **Robot** | placa + motores + controlador + sensores |

## Ejercicios

1. **Nombra tres diferencias entre Arduino UNO y micro:bit.**

2. **¿Qué hace este código: `digitalWrite(13, HIGH); delay(2000); digitalWrite(13, LOW); delay(2000);`?**

3. **Dibuja el circuito para encender un LED desde el pin 7.**

4. **¿Qué valor devolverá `analogRead` si la tensión es 2,5 V?**

5. **Modifica el «Blink» para que el LED parpadee 4 veces por segundo.**

6. **Escribe con bloques de MakeCode un termómetro que muestre la temperatura al pulsar A.**

7. **¿Por qué no se conecta un motor directamente a un pin de Arduino?**

8. **Explica qué es el hardware libre.**

9. **Diseña un juego de reflejos con micro:bit: gana quien pulse antes su botón cuando aparezca una imagen.**

10. **Describe un robot que esquive obstáculos: componentes y programa.**

## Practica

1. **Un microcontrolador es...**

   **A)** un tipo de sensor

   **B)** un ordenador pequeño en un solo chip

   **C)** un cable USB

2. **Arduino es un proyecto de...**

   **A)** software de pago

   **B)** una consola

   **C)** hardware libre

3. **En Arduino, la función que se ejecuta una sola vez es...**

   **A)** setup()

   **B)** loop()

   **C)** delay()

4. **delay(1000) espera...**

   **A)** 1 minuto

   **B)** 1 milisegundo

   **C)** 1 segundo

5. **analogRead devuelve en Arduino UNO un número entre...**

   **A)** 0 y 1

   **B)** 0 y 1023

   **C)** 0 y 100

6. **La resistencia en serie con un LED sirve para...**

   **A)** limitar la corriente

   **B)** aumentar la luz

   **C)** cambiar el color

7. **La pata larga del LED es el...**

   **A)** cátodo (–)

   **B)** ánodo (+)

   **C)** neutro

8. **¿Cuántos LED tiene la matriz de la micro:bit?**

   **A)** 16

**B)** 64

**C)** 25

9. **El editor web por bloques de la micro:bit es...**

**A)** MakeCode

**B)** Arduino IDE

**C)** Photoshop

10. **Para detectar que agitas la micro:bit se usa el...**

**A)** sensor de luz

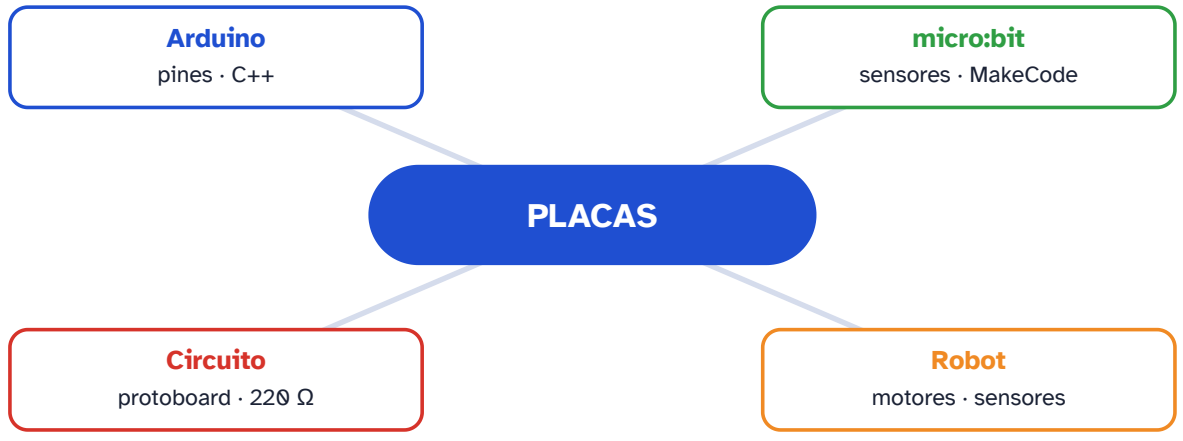
**B)** acelerómetro

**C)** botón B

## Explícalo con tus palabras

Incluye: Qué es un microcontrolador y en qué se diferencia de un ordenador; Cómo se monta un LED con su resistencia; Qué hacen setup() y loop(); Ventajas de la micro:bit para empezar.

## Resumen



## Tarjetas de repaso

Recórtalas: pregunta delante, respuesta detrás.

**Microcontrolador**

Ordenador en un chip que ejecuta un programa sin parar.

**Pin**

Patilla de la placa para conectar entradas o salidas.

**Protoboard**

Placa para montar circuitos sin soldar.

**setup()**

Se ejecuta una vez al encender: configuración.

**loop()**

Se repite sin fin.

**digitalWrite**

Pone un pin a HIGH (5 V) o LOW (0 V).

**analogRead**

Lee una tensión como número de 0 a 1023.

**MakeCode**

Editor por bloques de la micro:bit con simulador.

## Soluciones

1. **B)** Procesador, memoria y E/S.
2. **C)** Diseños públicos.
3. **A)** Configuración.
4. **C)** 1.000 ms.
5. **B)** 10 bits.
6. **A)** Evita que se queme.
7. **B)** Va hacia el positivo.
8. **C)** 5 × 5.
9. **A)** De Microsoft.
10. **B)** Movimiento e inclinación.

## Soluciones de los ejercicios

1. Por ejemplo: micro:bit tiene sensores integrados; Arduino necesita protoboard; micro:bit se programa con MakeCode y Arduino con su IDE en C++.
2. Enciende el LED del pin 13 dos segundos y lo apaga dos segundos, sin parar si está en loop().
3. Pin 7 → resistencia 220 Ω → pata larga del LED → pata corta → GND.
4. Aproximadamente 512 (la mitad de 1023).
5. Cada ciclo dura 250 ms: delay(125) encendido y delay(125) apagado.
6. al presionar botón A → mostrar número (temperatura (°C)).
7. Porque necesita más corriente de la que da un pin; se usa un controlador de motores y alimentación aparte.
8. Hardware cuyos diseños son públicos y pueden estudiarse, modificarse y fabricarse libremente.
9. Esperar un tiempo al azar; mostrar imagen; al presionar A → mostrar «A gana»; al presionar B → mostrar «B gana».
10. Placa, dos motores con controlador, batería, sensor de ultrasonidos; por siempre: si distancia < 15 cm, girar; si no, avanzar.